

Blockchain Experiment 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts

THEORY:

Q1: Primitive Data Types, Variables, Functions - pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual

data. Variables in Solidity can be

- **state variables:** stored on the blockchain permanently
- **local variables:** temporary, created during function execution
- **global variables:** special predefined variables such as msg.sender, msg.value, and block.timestamp

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

Q2: Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation.

For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

Q3: Visibility, Modifiers and Constructors

Function Visibility defines who can access a function:

- **public:** available both inside and outside the contract.
- **private:** only accessible within the same contract.
- **internal:** accessible within the contract and its child contracts.
- **external:** can be called only by external accounts or other contracts.

Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

Q4: Control Flow : if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops (for, while, do-while)** enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

Q5: Data Structures : Arrays, Mappings, structs, enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type. Example: struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

Q6: Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory.

Understanding data locations is essential, as they directly impact gas costs and performance.

Q7: Transactions : Ether and wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

TASKS PERFORMED:

Tutorial 1: Introduction

a. get

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. Under 'CONTRACT', 'Counter - remix-project-org/remix-w' is selected. The 'Deploy' button is orange, and 'At Address' is greyed out. Below, 'Transactions recorded' shows 7 transactions. 'Deployed Contracts' shows 'COUNTER AT 0x5E1...4EFF5 (M)' with a balance of 0 ETH. Buttons for 'dec', 'inc', 'count', 'get', and 'get - call' are visible. The 'get' button is highlighted. Below these, 'Low level interactions' shows 'CALLDATA' and a 'Transact' button. The main editor on the right shows the Solidity code for the 'Counter' contract. The code includes a 'get()' function that returns the current count. The bottom status bar shows 'Scan Alert' and 'Initialize as git repo'.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

b. inc

The screenshot shows the Remix IDE interface, similar to the previous one, but with the 'inc' button highlighted. The 'get' button is no longer highlighted. The 'Low level interactions' section shows 'CALLDATA' and a 'Transact' button. The main editor on the right shows the same Solidity code for the 'Counter' contract. The bottom status bar shows 'Scan Alert' and 'Initialize as git repo'.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

c. dec

REMIX 1.5.1

learnmeth tutorials

DEPLOY & RUN TRANSACTIONS

CONTRACT

Counter - remix-project-org/remix-w

em version: osaka

Deploy

At Address Load contract from Address

Transactions recorded 0

Deployed Contracts

COUNTER AT 0x5E1...4EFF5 (M)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 3

Low level interactions

CALLDATA

Transact

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

Explain contract

0 Listen on all transactions

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 2: Basic Syntax

REMIX 1.5.1

learnmeth tutorials

LEARNETH

Tutorials list

Syllabus

2. Basic Syntax 2 / 19

it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Explain contract

0 Listen on all transactions

CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 3: Primitive Data Types

remix.ethereum.org/#activate=udapp.sol⟨=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.f1a2265.js

REMUX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types 3 / 19

You can learn more about these data types as well as [Fixed Point Numbers](#), [Byte Arrays](#), [Strings](#), and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like [Mappings](#), [Arrays](#), [Enums](#), and [Structs](#).

Watch a video tutorial on Primitive Data Types.

★ Assignment

1. Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `new`, that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

Scan Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
20 // Negative numbers are allowed for int types.
21 // (like uint, different ranges are available from int8 to int256)
22 //
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xcA37957d915458f540a06040dfc2f4818fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultUint; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35
36 // New values
37 address public newAddr = 0x0000000000000000000000000000000000000000;
38 int public neg = -12;
39 uint8 public new = 0;
40 }
```

Explain contract

0 Listen on all transactions

[call] from: 0xc58380a6a701c568545d0cf803fc8875f56bedc4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 4: Variables

remix.ethereum.org/#activate=udapp.sol⟨=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.f1a2265.js

REMUX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

4. Variables 4 / 19

addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

Scan Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

Explain contract

0 Listen on all transactions

[call] from: 0xc58380a6a701c568545d0cf803fc8875f56bedc4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 5: Functions - Reading and Writing to a State Variable

LEARNETH 1.5.1

Tutorials list **Syllabus**

5.1 Functions - Reading and Writing to a State Variable 1/19

names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ **Assignment**

1. Create a public state variable called `num` that is of type `uint` and initialize it to `true`.
2. Create a public function called `get` that returns the value of `num`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

SimpleStorage.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public {
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) {
16        return num;
17    }
18
19    function get_b() public view returns (bool) {
20        return b;
21    }
22 }
```

Explain contract

0 Listen on all transactions

[call] from: 0x5B38Da7a701c568545dCfcB03Fc8B75F56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 6: Functions - View and Pure

LEARNETH 1.5.1

Tutorials list **Syllabus**

5.2 Functions - View and Pure 6/19

opcodes."

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ **Assignment**

Create a function called `addrox2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

ViewAndPure.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addrox(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addrox2(uint y) public {
18        x = x + y;
19    }
20 }
```

Explain contract

0 Listen on all transactions

[call] from: 0x5B38Da7a701c568545dCfcB03Fc8B75F56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 7: Functions - Modifiers and Constructors

remix.ethereum.org/#?activate=udapp.solidity.LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.f3a2265.js

REMUX 1.5.1 learneth-tutorials

LEARNETH

Tutorials list Syllabus

5.3 Functions - Modifiers and Constructors 7/19

parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increase` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increase` should be empty.

Tip: Use modifiers.

Check Answer Show answer

Next

Well done! No errors.

```
43 _;
44 x = x + y;
45 }
46
47 function increase(uint y) public onlyOwner biggerThan(y) increaseBy(y) {
48
49
50 // Modifiers can be called before and / or after a function.
51 // This modifier prevents a function from being called while
52 // it is still executing.
53 modifier noReentrancy() {
54     require(!locked, "No reentrancy");
55
56     locked = true;
57     _;
58     locked = false;
59 }
60
61 function decrement(uint i) public noReentrancy {
62     x -= i;
63
64     if (i > 1) {
65         decrement(i - 1);
66     }
67 }
68 }
```

Explain contract

Listen on all transactions

[call] from: 0x58380a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 8: Functions - Inputs and Outputs

remix.ethereum.org/#?activate=udapp.solidity.LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.f3a2265.js

REMUX 1.5.1 learneth-tutorials

LEARNETH

Tutorials list Syllabus

5.4 Functions - Inputs and Outputs 8/19

the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

```
64
65 return (i, b, j, x, y);
66 }
67
68 // Cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public {
72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80 function returnTwo()
81 public
82 pure
83 returns (
84     int i,
85     bool b
86 )
87 {
88     i = -2;
89     b = true;
90 }
91 }
```

Explain contract

Listen on all transactions

[call] from: 0x58380a701c568545dcfc803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Scan Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 9: Visibility

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list Syllabus

6. Visibility 9/19

- State variables can not be `external`.

In this example, we have two contracts, the `base` contract (line 4) and the `child` contract (line 55) which inherits the functions and state variables from the `base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `testPrivateFunc` from the `base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `testPrivateFunc` and `testInternalFunc` directly. You will only be able to call them via `testInternalFunc` and `testInternalVar`.

Watch a video tutorial on Visibility.

★ Assignment

Create a new function in the `base` contract called `testStateVar` that returns the values of all state variables from the `base` contract that are possible to return.

Check Answer Show answer

Next

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract base {
5     // State variables
6     string private privateVar = "my private variable";
7     string internal internalVar = "my internal variable";
8     string public publicVar = "my public variable";
9     // State variables cannot be external so this code won't compile.
10    // string external externalVar = "my external variable";
11 }
12
13 contract child is base {
14     // Inherited contracts do not have access to private functions
15     // and state variables.
16     // function testPrivateFunc() public pure returns (string memory) {
17     //     return privateVar();
18     // }
19
20     // Internal function call be called inside child contracts.
21     function testInternalFunc() public pure override returns (string memory) {
22         return internalFunc();
23     }
24
25     function testInternalVar() public view returns (string memory, string memory) {
26         return (internalVar, publicVar);
27     }
28 }
```

0 Listen on all transactions

[call] from: 0x58380a6a701c568545dcf803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 10: Control Flow - If/Else

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list Syllabus

7.1 Control Flow - If/Else 10/19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ Assignment

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer Show answer

Next

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint x) public pure returns (uint) {
16        // if (x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20
21        // shorthand way to write if / else statement
22        return x < 10 ? 1 : 2;
23    }
24
25    function evenCheck(uint y) public pure returns (bool) {
26        return y % 2 == 0 ? true : false;
27    }
28 }
```

0 Listen on all transactions

[call] from: 0x58380a6a701c568545dcf803fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Tutorial 11: Control Flow - Loops

The screenshot shows the Remix IDE interface for Tutorial 11: Control Flow - Loops. The sidebar on the left displays the tutorial list, with the current tutorial selected. The main editor shows the Solidity code for a contract named `loop`. The code includes a `uint public count;` and a `function loop() public` that uses a `for` loop and a `while` loop. The console at the bottom shows the transaction details for the `loop` function.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract loop {
5     uint public count;
6     function loop() public {
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                 // Skip to next iteration with continue
11                 continue;
12             }
13             if (i == 5) {
14                 // Exit loop with break
15                 break;
16             }
17             count++;
18         }
19
20         // while loop
21         uint j;
22         while (j < 10) {
23             j++;
24         }
25     }
26 }
27
```

Check Answer Show answer

Well done! No errors.

Tutorial 12: Data Structures - Arrays

The screenshot shows the Remix IDE interface for Tutorial 12: Data Structures - Arrays. The sidebar on the left displays the tutorial list, with the current tutorial selected. The main editor shows the Solidity code for a contract named `CompactArray`. The code includes a `uint[] public arr;` and a `function remove(uint index) public` that uses the `delete` operator to remove an element from the array. The console at the bottom shows the transaction details for the `remove` function.

```
40 // Delete does not change the array length.
41 // It resets the value at index to its default value,
42 // in this case 0
43 delete arr[index];
44 }
45
46
47 contract CompactArray {
48     uint[] public arr;
49
50     // Deleting an element creates a gap in the array.
51     // One trick to keep the array compact is to
52     // move the last element into the place to delete.
53     function remove(uint index) public {
54         // Move the last element into the place to delete
55         arr[index] = arr[arr.length - 1];
56         // Remove the last element
57         arr.pop();
58     }
59
60     function test() public {
61         arr.push(1);
62         arr.push(2);
63         arr.push(3);
64         arr.push(4);
65         // [1, 2, 3, 4]
66
67         remove(1);
68         // [1, 4, 3]
69
70         remove(2);
71         // [1, 4]
72     }
73 }

```

Check Answer Show answer

Well done! No errors.

Tutorial 13: Data Structures - Mappings

The screenshot shows the Remix IDE interface for Tutorial 13: Data Structures - Mappings. The left sidebar contains the 'LEARNETH' sidebar with a 'Tutorials list' and 'Syllabus' section. The 'Syllabus' section shows '8.2 Data Structures - Mappings' with a '13 / 19' progress indicator. The 'Tutorials list' section shows 'mapping's name and key in brackets and assigning it a new value (line 16)'. The 'Removing values' section explains that the delete operator can be used to delete a value associated with a key, which will set it to the default value of 0. The 'Assignment' section lists three steps: 1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`. 2. Change the functions `get` and `remove` to work with the mapping `balances`. 3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter. The 'Check Answer' and 'Show answer' buttons are visible. The 'Well done! No errors.' message is displayed at the bottom.

The central editor shows the Solidity code for the `mappings.sol` file. The code includes functions `set`, `remove`, and `get` for a `balances` mapping. It also includes a `NestedMapping` contract with a nested mapping `mapping(address => mapping(uint => bool))` and functions `set` and `remove` for it.

```
11 }
12
13
14 function set(address _addr) public { 2536 gas
15     // update the value at this address
16     balances[_addr] = _addr.balance;
17 }
18
19 function remove(address _addr) public { 2536 gas
20     // Reset the value to the default value.
21     delete balances[_addr];
22 }
23
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr1, uint _i) public view returns (bool) { 2536 gas
30         // You can get values from a nested mapping
31         // even when it is not initialized
32         return nested[_addr1][_i];
33     }
34
35     function set( 2536 gas
36         address _addr1,
37         uint _i,
38         bool _boo
39     ) public {
40         nested[_addr1][_i] = _boo;
41     }
42
43     function remove(address _addr1, uint _i) public { 25845 gas
44         delete nested[_addr1][_i];
45     }
46 }
```

Tutorial 14: Data Structures - Structs

The screenshot shows the Remix IDE interface for Tutorial 14: Data Structures - Structs. The left sidebar contains the 'LEARNETH' sidebar with a 'Tutorials list' and 'Syllabus' section. The 'Syllabus' section shows '8.3 Data Structures - Structs' with a '14 / 19' progress indicator. The 'Tutorials list' section shows 'parameters in parentheses (line 16)'. The 'Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19)'. The 'Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23)'. The 'Accessing structs' section explains that to access a member of a struct we can use the dot operator (line 33). The 'Updating structs' section explains that to update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45). The 'Assignment' section lists three steps: 1. Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping. 2. Create a function `toggleCompleted` that takes a `uint` as a parameter and toggles the `completed` member of the struct at the given index in the `todos` mapping. 3. Create a function `update` that takes a `uint` as a parameter and updates the `text` member of the struct at the given index in the `todos` mapping. The 'Check Answer' and 'Show answer' buttons are visible. The 'Well done! No errors.' message is displayed at the bottom.

The central editor shows the Solidity code for the `structs.sol` file. The code includes a `Todos` struct with `text` and `completed` members. It includes functions `push`, `get`, `update`, `toggleCompleted`, and `remove` for the `todos` array.

```
16 todos.push(Todos(text, completed));
17
18 // key value mapping
19 todos.push(Todos({text: _text, completed: false}));
20
21 // initialize an empty struct and then update it
22 Todos memory todo;
23 todo.text = _text;
24 // todo.completed initialized to false
25
26 todos.push(todo);
27
28
29 // Solidity automatically created a getter for 'todos' so
30 // you don't actually need this function.
31 function get(uint _index) public view returns (string memory text, bool completed)
32 {
33     Todos storage todo = todos[_index];
34     return (todo.text, todo.completed);
35 }
36
37 // update text
38 function update(uint _index, string memory _text) public { Infinite gas
39     Todos storage todo = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleCompleted(uint _index) public { 28995 gas
45     Todos storage todo = todos[_index];
46     todo.completed = !todo.completed;
47 }
48
49 function remove(uint _index) public { Infinite gas
50     delete todos[_index];
51 }
```

Tutorial 15: Data Structures - Enums

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list

8.4 Data Structures - Enums 15 / 19

We can update the enum value of a variable by assigning it the `value` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `size` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `size`.

Check Answer Show answer

Next

Well done! No errors.

Explain contract

```
13
14
15 enum Size {
16     S,
17     M,
18     L
19 }
20
21 // Default value is the first element listed in
22 // definition of the type, in this case "Pending"
23 Status public status;
24 Size public sizes;
25
26 function get() public view returns (Status) {
27     return status;
28 }
29
30 function getSize() public view returns (Size) {
31     return sizes;
32 }
33
34 // Update status by passing uint into input
35 function set(status _status) public {
36     status = _status;
37 }
38
39 // You can update to a specific enum like this
40 function cancel() public {
41     status = Status.Canceled;
42 }
43
44 // delete resets the enum to its first value, 0
45 function reset() public {
46     delete status;
47 }
```

Tutorial 16: Data Locations

REMUX 1.5.1 learneth tutorials

LEARNETH

Tutorials list

9. Data Locations 16 / 19

amount of gas possible.

★ **Assignment**

1. Change the value of the `myStruct` member `foo`, inside the function `f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

Explain contract

```
13
14
15 function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {
16     // call f with state variables
17     f(arr, map, myStructs[1]);
18     // get a struct from a mapping
19     MyStruct storage myStruct = myStructs[1];
20     myStruct.foo = 4;
21     // create a struct in memory
22     MyStruct memory myMemStruct = MyStruct(0);
23     MyStruct memory myMemStruct2 = myMemStruct;
24     myMemStruct2.foo = 1;
25
26     MyStruct memory myMemStruct3 = myStruct;
27     myMemStruct3.foo = 3;
28     return (myStruct, myMemStruct2, myMemStruct3);
29 }
30
31 function f(
32     uint[] storage _arr,
33     mapping(uint => address) storage _map,
34     MyStruct storage _myStruct
35 ) internal {
36     // do something with storage variables
37
38     // You can return memory variables
39     function g(uint[] memory _arr) public returns (uint[] memory) {
40         // do something with memory array
41         _arr[0] = 1;
42     }
43
44     function h(uint[] calldata _arr) external {
45         // do something with calldata array
46         _arr[0] = 1;
47     }
48 }
```

Tutorial 17: Transactions - Ether and Wei

LEARNETH 15.1

Tutorials list 17 / 19

Wei is the smallest subunit of Ether, named after the cryptographer Wei Dai. Ether numbers without a suffix are treated as `wei` (line 7).

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^{10}) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneWei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

etherAndWei.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract etherAndWei {
5     uint public oneWei = 1 gwei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 18: Transactions - Gas and Gas Price

LEARNETH 15.1

Tutorials list 18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about [gas](#) on [ethereum.org](#).

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

gasAndGasPrice.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

Tutorial 19: Transactions - Sending Ether

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar is visible, showing a list of assignments. The main editor displays the Solidity code for a 'Charity' contract. The code includes a constructor, a 'donate' function, and a 'withdraw' function. The 'withdraw' function uses 'owner.call' to send ether to the beneficiary.

```
34 // This function is no longer recommended for sending ether.
35 // to.transfer(msg.value);
36 }
37
38 function sendViaSend(address payable _to) public payable { // undefined gas
39     // Send returns a boolean value indicating success or failure.
40     // This function is not recommended for sending ether.
41     bool sent = _to.send(msg.value);
42     require(sent, "Failed to send Ether");
43 }
44
45 function sendViaCall(address payable _to) public payable { // undefined gas
46     // Call returns a boolean value indicating success or failure.
47     // This is the current recommended method to use.
48     (bool sent, bytes memory data) = _to.call{value: msg.value}("");
49     require(sent, "Failed to send Ether");
50 }
51
52
53 contract Charity {
54     address public owner;
55
56     constructor() { // 305432 gas, 142888 gas
57         owner = msg.sender;
58     }
59
60     function donate() public payable {} // 141 gas
61
62     function withdraw() public { // infinite gas
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call{value: amount}("");
66         require(sent, "Failed to send Ether");
67     }
68 }
```

CONCLUSION:

Through this experiment, the basic concepts of Solidity programming were learned by completing practical assignments using the Remix IDE. Important topics such as data types, variables, different types of functions, visibility, modifiers, constructors, control flow statements, data structures, and transactions were studied and applied while creating smart contracts. The hands-on practice helped in understanding how to design, compile, and deploy contracts using the Remix VM. Overall, this experiment helped in building a clear understanding of blockchain concepts and provided a strong foundation for developing and managing smart contracts effectively.